

文章笔记

文章笔记：Harness Engineering

基于 Martin Fowler 博客文章整理

2026 年 4 月 2 日

文章作者：Birgitta Böckeler (Thoughtworks)

发布日期：2026-02-17

预计阅读时长：约 10 分钟

文 章 链 接：<https://martinfowler.com/articles/exploring-gen-ai/harness-engineering.html>

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 引言：什么是 Harness Engineering	2
1.1 本章小结	2
2 Harness 的三大组成部分	3
2.1 Context Engineering（上下文工程）	3
2.2 Architectural Constraints（架构约束）	3
2.3 Garbage Collection（“垃圾回收”）	4
2.4 迭代反馈循环	4
2.5 Birgitta 指出的缺失部分	4
2.6 本章小结	4
3 Harness 作为未来的服务模板	5
3.1 从 Service Template 到 Harness Template	5
3.2 分叉与同步挑战	5
3.3 本章小结	5
4 约束运行时以换取 AI 自主性	6
4.1 “生成任意代码”的幻觉	6
4.2 约束作为赋能	6
4.3 本章小结	6
5 技术栈收敛与 AI 友好性	7
5.1 从“开发者品味”到“AI 友好性”	7
5.2 代码库拓扑作为新的抽象层	7
5.3 本章小结	7
6 Pre-AI 与 Post-AI 的两个世界	8
6.1 新建应用 vs 遗留系统	8
6.2 两个世界的分化	8
6.3 本章小结	8
7 今天你的 Harness 是什么？	9
7.1 渐进式构建 Harness 的起点	9
7.2 从人类实践到 AI 实践的连续性	9
7.3 本章小结	9
8 深层思考：严谨性的迁移	10
8.1 超越 Markdown 规则文件	10
8.2 “严谨性的迁移”（Relocating Rigor）	10
8.3 本章小结	10

9 总结与延伸	11
9.1 核心观点回顾	11
9.2 对实践者的启示	11
9.3 拓展阅读	11

1 引言：什么是 Harness Engineering

2026 年 2 月, OpenAI 发表了一篇题为 “Harness engineering: leveraging Codex in an agent-first world” 的文章, 描述了一个团队如何在**完全不手动编写代码**的前提下, 利用 AI agent 维护一个超过 100 万行代码的大型应用。这一实践持续了 5 个月, 产出了一个真实的产品。Thoughtworks 的 Distinguished Engineer Birgitta Böckeler 在 Martin Fowler 的博客上撰文, 对这篇文章进行了深入的架构视角分析。

Harness 的核心定义

“Harness” (直译为 “缰绳” 或 “线束”) 在这里指的是一整套**工具、实践和约束**, 用于在 AI agent 驱动的软件开发中保持代码质量和可维护性。它不是单一工具, 而是一个**系统性的控制框架**, 结合了确定性规则和 LLM 驱动的检查机制。

值得注意的是, “harness” 这个术语可能受到了 Mitchell Hashimoto 近期博客文章的启发——OpenAI 的文章标题包含这个词, 但正文中仅出现了一次。无论来源如何, Birgitta 认为 “harness” 是一个非常恰当的词, 用来描述我们可以用来 “约束” AI agent 的工具和实践。

文章背景

本文是 Birgitta Böckeler 在 Martin Fowler 博客上 “Exploring Gen AI” 系列的一部分。Birgitta 拥有超过 20 年的软件开发、架构和技术领导经验, 目前专注于 AI 辅助交付 (AI-assisted delivery) 领域。她的分析融合了丰富的工程实践经验与对 AI 工具的深度思考。

1.1 本章小结

Harness engineering 是一种新兴的工程实践, 旨在通过系统性的工具和约束来管控 AI agent 的代码生成行为。它的出现标志着 AI 辅助编程正从 “让 AI 生成任意代码” 的阶段, 走向 “有控制地利用 AI 进行大规模软件维护” 的阶段。

2 Harness 的三大组成部分

Birgitta 将 OpenAI 团队描述的 harness 组件归纳为三个类别，这些组件混合使用了确定性 (deterministic) 和基于 LLM 的方法。

2.1 Context Engineering (上下文工程)

第一个组成部分是上下文工程，即持续增强代码库中的知识库，同时让 agent 能够访问动态上下文信息。

Context Engineering 的两个维度

1. **静态知识库**：在代码库中持续维护和增强的文档、设计决策记录、架构约定等
2. **动态上下文**：agent 可以实时访问的可观测性数据 (observability data)、浏览器导航等运行时信息

上下文工程不仅仅是编写 Markdown 规则文件那么简单——代码设计本身就是上下文的重要组成部分。

这一点非常重要：OpenAI 团队的上下文工程不仅涉及知识库的整理 (curation)，还涉及大量的设计工作——代码的设计本身就是上下文的核心组成部分。这意味着，好的代码架构不仅对人类开发者有益，对 AI agent 同样至关重要。

2.2 Architectural Constraints (架构约束)

第二个组成部分是**架构约束**，通过确定性和 LLM 双重机制进行监控和执行。
具体包括：

- **基于 LLM 的 agent 监控**：AI agent 在生成代码时自我检查是否符合架构约定
- **确定性自定义 linter**：不依赖 AI 的硬性规则检查工具
- **结构性测试 (structural tests)**：验证代码结构是否符合预期的自动化测试

确定性 vs LLM 驱动的检查

在 harness 中，确定性工具（如自定义 linter、结构性测试）和 LLM 驱动的检查各有优势：

维度	确定性工具	LLM 驱动
可靠性	100% 一致	可能有误判
灵活性	仅处理预定义模式	可理解语义意图
维护成本	需手动更新规则	可自适应新模式
适用场景	硬性边界、格式规范	设计意图、模式匹配

两者的结合是 harness 设计的关键——确定性工具提供可靠的底线保障，LLM 则覆盖更复杂的语义检查。

2.3 Garbage Collection (“垃圾回收”)

第三个组成部分是**周期性运行的清理 agent**，负责发现文档中的不一致性或架构约束的违规行为，对抗代码库的**熵增**（entropy）和**衰减**（decay）。

代码库的熵增问题

在大规模代码库中，即使每次单独的修改都是合理的，随着时间推移，整体代码质量也会不断下降——这就是软件工程中的“熵增”现象。传统团队依靠代码审查和定期重构来对抗熵增；在 AI 驱动的开发中，这种熵增可能会更快发生，因此需要专门的“garbage collection” agent 来持续清理和修复。

2.4 迭代反馈循环

OpenAI 团队特别强调了 harness 的**迭代性**。他们的核心方法论可以概括为：

Harness 的迭代改进方法论

“当 agent 遇到困难时，我们将其视为一个信号：识别缺失的内容——工具、护栏、文档——并将其反馈回代码库，始终由 Codex 本身来编写修复。”

这形成了一个**正反馈循环**：agent 的失败驱动 harness 的改进，而改进后的 harness 又提升了 agent 的能力。

2.5 Birgitta 指出的缺失部分

Birgitta 注意到，OpenAI 文章中描述的所有措施都集中在提升长期的**内部质量**（internal quality）和**可维护性**（maintainability），但缺少对**功能和行为验证**（verification of functionality and behaviour）的讨论。这是一个值得关注的 gap——代码结构良好但功能不正确，同样是不可接受的。

内部质量 vs 外部质量

OpenAI 的 harness 聚焦于架构约束、代码一致性等**内部质量**指标，但没有讨论如何验证代码的**外部质量**——即功能是否正确、行为是否符合预期。一个完整的 harness 应该同时覆盖这两个维度。缺少功能验证的 harness 就像只检查语法不检查语义的编译器。

2.6 本章小结

Harness 由三大组成部分构成：上下文工程提供 AI 所需的知识和信息；架构约束通过确定性和 LLM 双重机制执行规则；“垃圾回收” agent 持续对抗代码库的熵增。三者的迭代协作形成正反馈循环，但目前在功能验证方面仍有缺口。

3 Harness 作为未来的服务模板

3.1 从 Service Template 到 Harness Template

Birgitta 提出了一个引人深思的观察：大多数组织只有**两到三个主要技术栈**——并非每个应用都是独特的“雪花”（snowflake）。这让她设想了一个未来场景：团队可以从一组针对**常见应用拓扑**（common application topologies）的 harness 中进行选择来启动项目。

这个设想直接对应了当今软件工程中的 **service template**（服务模板）概念。Service template 帮助团队在“黄金路径”（golden path）上实例化新服务，提供标准化的项目结构、CI/CD 配置、监控集成等。

Service Template 的演进

维度	传统 Service Template	未来 Harness Template
包含内容	项目结构、配置、CI/CD	自定义 linter、结构性测试、知识文档、上下文提供者
目标用户	人类开发者	AI agent + 人类
初始化	一次性脚手架	持续演进的约束系统
维护方式	手动更新、回馈上游	agent 驱动的迭代改进

3.2 分叉与同步挑战

Birgitta 敏锐地指出了一个问题：在当前的 service template 实践中，团队基于模板起步后会根据自身需求进行定制，然后尝试将改进回馈给上游模板——但其他团队往往难以整合这些更新。这种**分叉与同步**的挑战在 harness 模式下是否会重现？

Harness 的分叉风险

当每个团队都基于通用 harness 模板进行定制时，可能出现以下问题：

- 不同团队的 harness 逐渐分化，失去统一标准的意义
- 将特定项目的改进合并回通用模板变得困难
- 通用模板的更新难以推送到已高度定制的项目 harness 中
- 团队可能陷入“harness 债务”——维护 harness 本身的负担越来越重

这与 service template 面临的问题本质相同，但由于 harness 包含更多动态组件（如 LLM 驱动的检查逻辑），分叉后的同步可能更加困难。

3.3 本章小结

Harness 有可能成为 service template 的进化形态，为 AI 驱动的开发提供标准化的控制框架。但这也意味着它将继承 service template 的分叉与同步挑战，甚至可能因为 harness 的动态特性而变得更加复杂。组织需要在标准化与定制化之间找到平衡。

4 约束运行时以换取 AI 自主性

4.1 “生成任意代码”的幻觉

AI 编程领域的早期炒作（以及当前的部分宣传）假设 LLM 将赋予我们**无限的运行时灵活性**——用任何语言、任何模式、不受约束地生成代码，AI 自己会搞定一切。

然而，OpenAI 团队的 harness 实践揭示了一个相反的事实：要实现可信赖的、大规模 AI 生成代码的可维护性，**必须有所取舍**。具体来说，提升信任度和可靠性的方式是**约束解决方案空间**——通过特定的架构模式、强制的边界、标准化的结构来限制 AI 的自由度。

灵活性与可维护性的权衡

核心洞察：为了获得可信赖的 AI 生成代码，我们需要用 prompt、规则和充满技术细节的 harness 来**放弃**“生成任意代码”的灵活性。这意味着：

- 更少的技术选择自由
- 更严格的架构约定
- 更详细的规范文档
- 但换来更高的代码质量和可维护性

这是一个经典的工程权衡：约束产生秩序，秩序产生信任。

4.2 约束作为赋能

这种“约束带来自主性”的思路在软件工程中并不新鲜。容器化（Docker/Kubernetes）通过约束运行环境来实现部署自由；微服务通过约束模块边界来实现独立开发和部署；类型系统通过约束数据表达来减少运行时错误。

在 AI 编程语境中，harness 的约束同样是一种**赋能**——通过限制 AI 的选择空间，反而让它在受限范围内表现得更好、更可靠。

4.3 本章小结

“生成任意代码”的无限灵活性是一个幻觉。实践证明，约束解决方案空间是获得可信赖 AI 生成代码的必要条件。这并非 AI 的局限，而是工程学的普遍规律——有意义的约束是高质量输出的前提。

5 技术栈收敛与 AI 友好性

5.1 从“开发者品味”到“AI 友好性”

Birgitta 提出了一个大胆的预测：随着编程从“手动编写代码”转向“引导代码生成”，AI 可能推动我们走向**更少的技术栈**。

她观察到，框架和 SDK 的易用性（usability）仍然重要——反复的实践表明，**对人类友好的东西对 AI 也友好**。但在具体细节层面，**开发者的个人品味将变得不那么重要**。接口中的小低效和特殊习惯不再那么烦人，因为我们不再直接与之打交道。

AI 驱动的技术栈选择标准变迁

在传统开发中，技术栈选择往往受以下因素影响：

- 团队熟悉度和开发者偏好
- 生态系统成熟度和社区活跃度
- 性能特征和部署便利性
- API 设计的人体工学（ergonomics）

在 AI 驱动的未来中，新的选择标准可能包括：

- 是否有高质量的 harness 模板可用
- 代码模式是否容易被 AI 理解和生成
- 架构约束是否容易通过工具实施
- “AI 友好性”——AI 在该技术栈上的生成质量

5.2 代码库拓扑作为新的抽象层

Birgitta 提出了一个更深层的问题：如果我们能够广泛地 harness 代码库设计模式，那么这些**拓扑结构**（topologies）是否会成为新的抽象层——而非许多 AI 爱好者所期望的**自然语言**？

OpenAI 团队讨论了**架构刚性**（architectural rigidity）和**强制规则**（enforcement rules），主要关注两个领域：

- **保持数据结构稳定**：确保核心数据模型不被随意修改
- **定义和强制模块边界**：确保模块之间的接口清晰且稳定

Birgitta 承认这些听起来合理，但缺乏具体示例让她难以想象 “we require Codex to parse data shapes at the boundary” 在实践中具体是什么样子。

5.3 本章小结

AI 可能推动技术栈的收敛——团队将优先选择有良好 harness 支持的技术栈。代码库的拓扑结构可能取代自然语言，成为 AI 编程时代的关键抽象层。这一趋势将重塑我们选择和评估技术的方式。

6 Pre-AI 与 Post-AI 的两个世界

6.1 新建应用 vs 遗留系统

Birgitta 提出了一个关键的实践问题：假设我们开发出了良好的 harness 技术，能够将 AI 自主性提升到很高水平并对结果充满信心——哪些技术可以应用于**现有应用**，哪些只适用于**从零开始**、从一开始就考虑 harness 的应用？

遗留代码库的 Harness 改造陷阱

对于较旧的代码库，需要慎重考虑改造（retrofit）harness 是否值得投入。虽然 AI 可以帮助加速改造过程，但这些遗留应用通常具有以下特征：

- 高度非标准化的代码结构
- 长期积累的技术债务和熵增
- 缺乏文档和架构约定
- 隐式依赖和未记录的假设

这就像在一个从未使用过静态代码分析工具的代码库上首次运行分析——你会被大量告警淹没。

6.2 两个世界的分化

这一观察暗示了未来可能出现**两种截然不同的软件维护模式**：

维度	Pre-AI 应用	Post-AI 应用
代码生成	主要由人类编写	主要由 AI agent 生成
架构约束	隐式、基于约定	显式、由 harness 强制
维护方式	人工审查 + 重构	agent 驱动 + 自动清理
Harness 适配	需要昂贵的改造	从设计之初内建
技术债务	大量历史遗留	由“垃圾回收”agent 持续清理

对于组织而言，这意味着需要制定**差异化的策略**：对新项目采用 harness-first 的方法，对遗留系统则需要评估改造的 ROI（投资回报率）。

6.3 本章小结

harness 技术在新建应用和遗留系统上的适用性存在本质差异。组织可能面临“两个世界”的分化：harness-native 的新应用和难以改造的旧应用。对遗留系统的 harness 改造需要谨慎评估投入产出比。

7 今天你的 Harness 是什么？

7.1 渐进式构建 Harness 的起点

OpenAI 团队花了 5 个月来构建他们的 harness——这不是一蹴而就的事情。但 Birgitta 鼓励每个团队反思自己当前的 harness 是什么，并提出了一系列引导性问题：

Harness 自检清单

以下问题可以帮助你评估当前团队的 harness 成熟度：

1. 你有 pre-commit hook 吗？里面包含了什么检查？
2. 你有自定义 linter 的想法吗？有哪些项目特定的规则想要自动化？
3. 你想在代码库上施加哪些架构约束？模块边界是否清晰？
4. 你尝试过结构性测试框架吗？比如 ArchUnit 这样验证架构规则的工具？

即使你还没有使用 AI agent 进行开发，这些问题本身就代表了良好的工程实践。

7.2 从人类实践到 AI 实践的连续性

这个自检清单揭示了一个重要的事实：harness engineering 的许多组件——linter、结构性测试、架构约束——本质上就是**良好的软件工程实践**。AI agent 的出现并没有发明这些概念，而是赋予了它们新的紧迫性和重要性。

在传统开发中，我们可以依靠经验丰富的开发者通过代码审查来执行隐式的架构约定。但当 AI agent 成为主要的代码生产者时，这些约定必须被**显式化**和**自动化**——因为 agent 无法理解“这里我们团队一般不这么做”这样的隐性知识。

ArchUnit 和结构性测试

ArchUnit 是一个用于检查 Java/Kotlin 代码架构规则的测试框架。它允许你编写类似以下的规则：“所有 Service 类不应依赖 Controller 类”、“所有 Repository 接口必须位于 persistence 包中”等。类似的工具在其他语言中也有对应：

- Java/Kotlin: ArchUnit
- .NET: NetArchTest
- Python: import-linter
- TypeScript: eslint-plugin-boundaries

这些工具是构建 harness 架构约束层的重要基础。

7.3 本章小结

构建 harness 是一个渐进的过程，可以从评估现有的 pre-commit hook、linter 和架构约束开始。harness 的核心组件本质上是良好工程实践的显式化和自动化。即使尚未使用 AI agent，建立这些实践也能提升团队的工程质量。

8 深层思考：严谨性的迁移

8.1 超越 Markdown 规则文件

Birgitta 指出，OpenAI 团队描述的工作远比“生成和维护一堆 Markdown 规则文件”要复杂得多。他们为 harness 的确定性部分构建了大量的工具，他们的上下文工程不仅涉及知识库的整理，还涉及大量的设计工作。

这一观察击中了当前 AI 编程工具使用中的一个常见误区：许多团队认为编写‘cursorsrules’或类似的 Markdown 规则文件就足够了。但 OpenAI 的实践表明，真正有效的 harness 需要工程投入 (engineering effort)，而非仅仅是提示工程 (prompt engineering)。

8.2 “严谨性的迁移” (Relocating Rigor)

OpenAI 团队表示：“我们目前最困难的挑战集中在设计环境、反馈循环和控制系统上。”这让 Birgitta 联想到了 Chad Fowler 最近关于“Relocating Rigor”（严谨性的迁移）的文章。

严谨性的迁移

在 AI 辅助编程时代，工程严谨性并没有消失，而是从一个层面迁移到了另一个层面：

- **之前：**严谨性体现在手动编写代码——命名规范、设计模式、代码审查
- **之后：**严谨性迁移到设计环境、反馈循环和控制系统——即 harness 的构建和维护

这不是严谨性的减少，而是严谨性的重新定位。听到关于“严谨性应该去向何处”的具体想法和经验，比仅仅寄希望于“更好的模型”会神奇地解决可维护性问题要令人振奋得多。

8.3 本章小结

有效的 harness 需要真正的工程投入，而非仅靠提示工程。工程严谨性从手动编码层面迁移到了环境设计、反馈循环和控制系统的构建上。这种“严谨性的迁移”是 AI 编程时代最重要的认知转变之一。

9 总结与延伸

9.1 核心观点回顾

Birgitta Böckeler 对 OpenAI harness engineering 实践的分析揭示了 AI 辅助编程的多个深层趋势：

1. **Harness 是系统性的**：由上下文工程、架构约束和“垃圾回收”三大组件构成，结合确定性和 LLM 驱动的方法
2. **约束产生信任**：放弃“生成任意代码”的灵活性，换取可维护性和可靠性
3. **技术栈将收敛**：AI 友好性可能成为技术选型的重要标准
4. **两个世界的分化**：新建应用可以 harness-native，遗留系统面临改造挑战
5. **严谨性在迁移**：工程严谨性从编码层迁移到环境和控制系统设计层
6. **功能验证仍有缺口**：当前的 harness 实践偏重内部质量，外部质量验证有待加强

9.2 对实践者的启示

保持批判性思维

OpenAI 对自身产品的成功案例描述需要保持审慎态度——作为 Codex 的开发者，他们在让我们相信 AI 可维护代码方面有天然的利益动机（vested interest）。在参考其实践时，应关注方法论本身的合理性，而非仅仅依赖其宣称的成功指标。

对于希望开始 harness engineering 实践的团队，以下是建议的优先级：

1. **评估现状**：盘点现有的 linter、pre-commit hook、CI 检查等
2. **显式化架构约定**：将隐式的团队约定转化为可自动化的规则
3. **引入结构性测试**：使用 ArchUnit 等工具验证架构边界
4. **建设知识库**：维护代码库中的设计决策记录和架构文档
5. **迭代改进**：将 AI agent 的失败视为 harness 改进的信号

9.3 拓展阅读

- OpenAI: “Harness engineering: leveraging Codex in an agent-first world”
- Mitchell Hashimoto: Harness 概念的早期讨论
- Chad Fowler: “Relocating Rigor”——严谨性迁移的思考
- Martin Fowler 博客 “Exploring Gen AI” 系列
- ArchUnit 官方文档——结构性测试框架
- Birgitta Böckeler: 原文链接